

DSA STUDY BLUEPRINT SERIES

74. Implement Queue using Stack Stack using Queue

Converting Stack LIFO to Queue FIFO and Vice-Versa

Section 08 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Queue using Stacks:** Uses two stacks. Transfer elements to reverse LIFO order.
- **Stack using Queue:** Rotate elements inside a single queue after every push operation.

Memory & Execution Blueprint

Stack Push(3) $\Rightarrow$ Queue: [1, 2] $\rightarrow$ Push 3 $\Rightarrow$ [1, 2, 3] $\rightarrow$
Rotate: [3, 1, 2]

C++ Implementation Reference

Standard code layout and syntax

```
class StackUsingQueue {  
    queue q;  
public:  
    void push(int x) {  
        q.push(x);  
        for(int i=0; i
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Push (Stack via Q)	$O(N)$ time, $O(N)$ space
Pop (Stack via Q)	$O(1)$

Key Practices & Gotchas

- **Important:** Using two queues yields similar conversions but requires extra memory allocation.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace circular queue pointer wrapping when capacity = 3:

Operation	Queue data layout	Pointers [front, rear]	Size count
enqueue(10)	[10, _, _]	[0, 0]	1
enqueue(20)	[10, 20, _]	[0, 1]	2
dequeue()	[_ , 20, _]	[1, 1]	1 (wraps correctly)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Wrap-around indices:** Modulo indexing updates circular buffer limits dynamically.
- **Deque mechanics:** Doubly ended queues allow fast $O(1)$ front and rear pushes and pops.
- **Related LeetCode Problems:** #239 Sliding Window Maximum, #622 Design Circular Queue, #225 Implement Stack using Queues.

